Introduction to Scientific Programming

Lecture 4: *Numerical Computing Foundations*

Renato Duarte

@Renatocfduarte
comp-neuro.org
@rcfduarte
renato.duarte@cnc.uc.pt

CENTER FOR NEUROSCIENCE AND CELL BIOLOGY
UNIVERSITY OF COIMBRA
PORTUGAL

1 2 9 0 UNIVERSIDADE D COIMBRA

Part I: *Fundamentals*

1. Introduction & Modern Development Ecosystem
2. Programming Fundamentals
3. Development Tools & Workflows

Part II: *Scientific Computing Core*

1. Numerical Computing Foundations (`numpy`, `scipy`, `matplotlib`)
2. Data Manipulation & Analysis (`pandas`, `scipy`)
3. Visualization & Communication (`matplotlib`, `seaborn`, `plotly`)
4. Intermediate Programming Concepts

Part II: *Scientific Computing Core*

Day 4: Numerical Computing Foundations

Lecture:

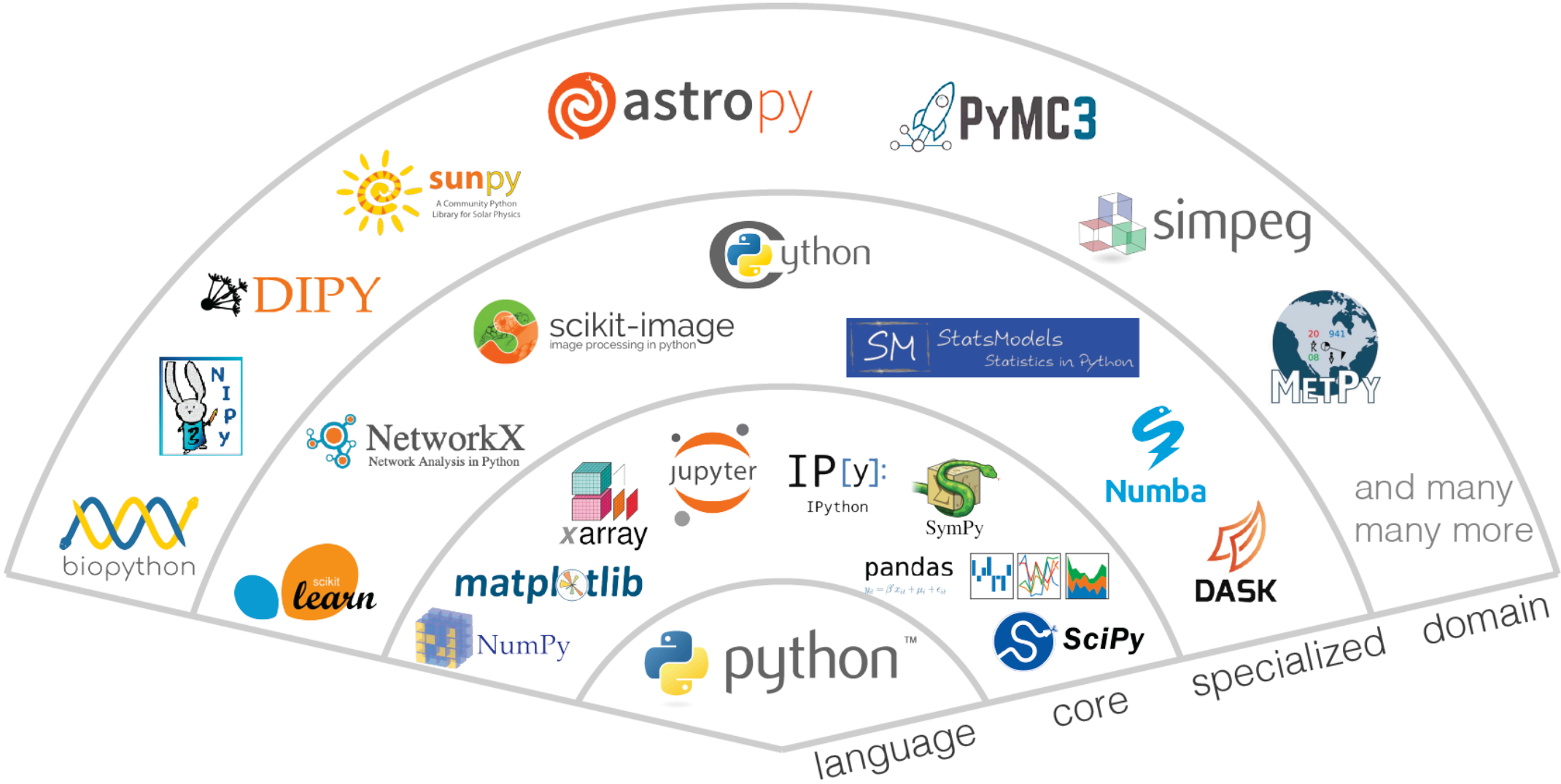
1. Numpy Fundamentals
    - What is numpy and why do we need it?
    - Numpy arrays: creation, indexing and operations
    - Basic functionality
2. Introduction to SciPy
    - The SciPy ecosystem and its components
    - Numpy + SciPy
3. Data visualization with Matplotlib

Practical:

- Handout & Tutorial exercises

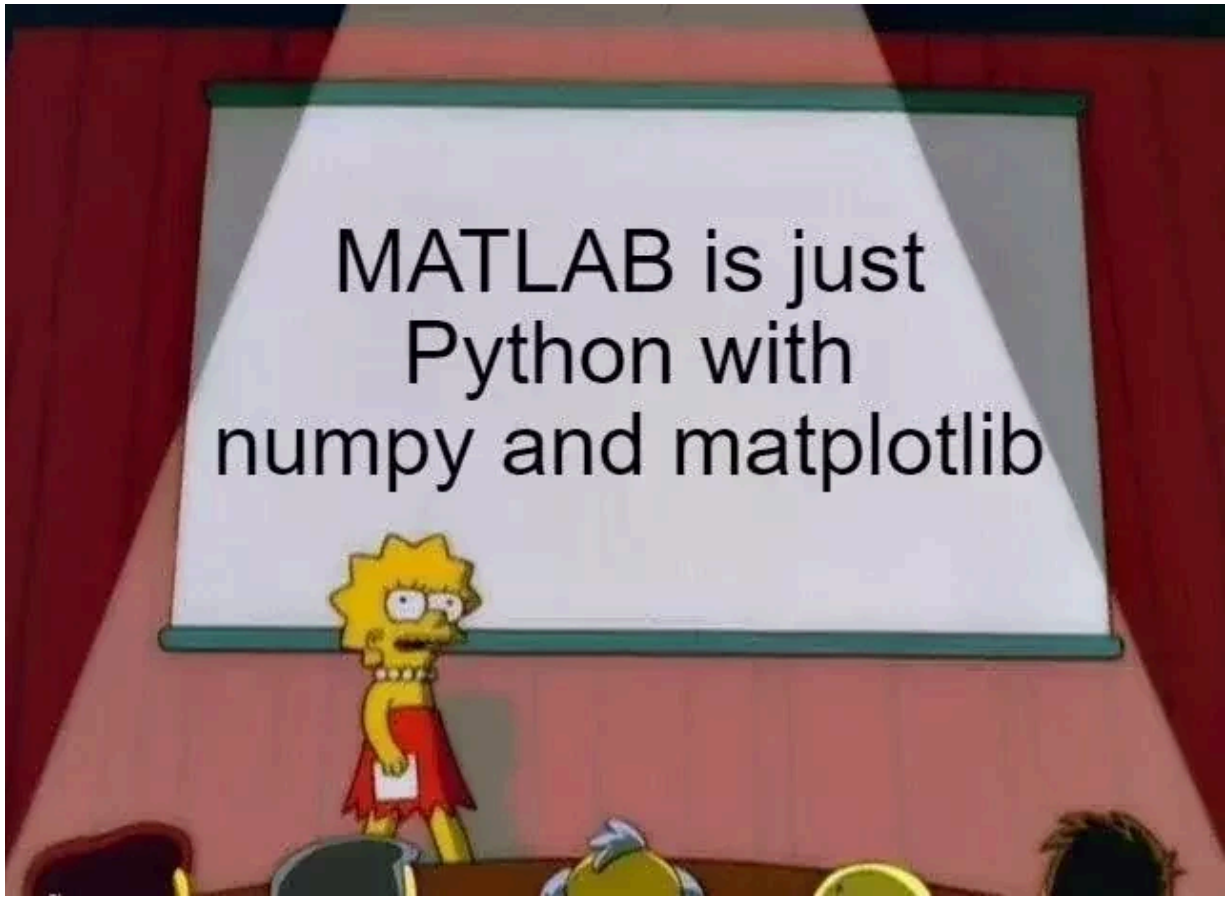
The Scientific Python Ecosystem

Building blocks for scientific computing



These three libraries form the foundation of scientific computing in Python

The Scientific Python Ecosystem



Building blocks for scientific computing

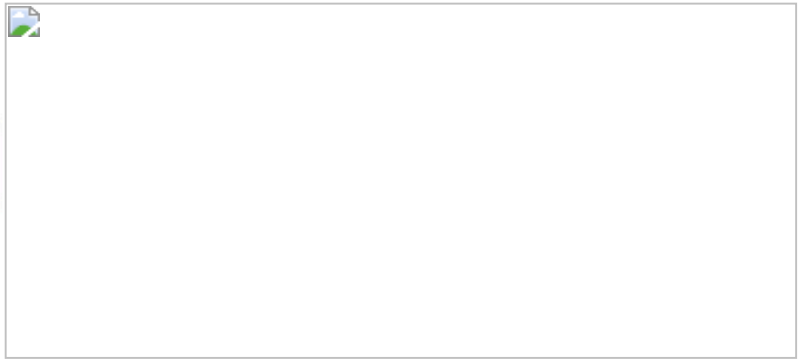
matplotlib

These three libraries form the foundation of scientific computing in Python

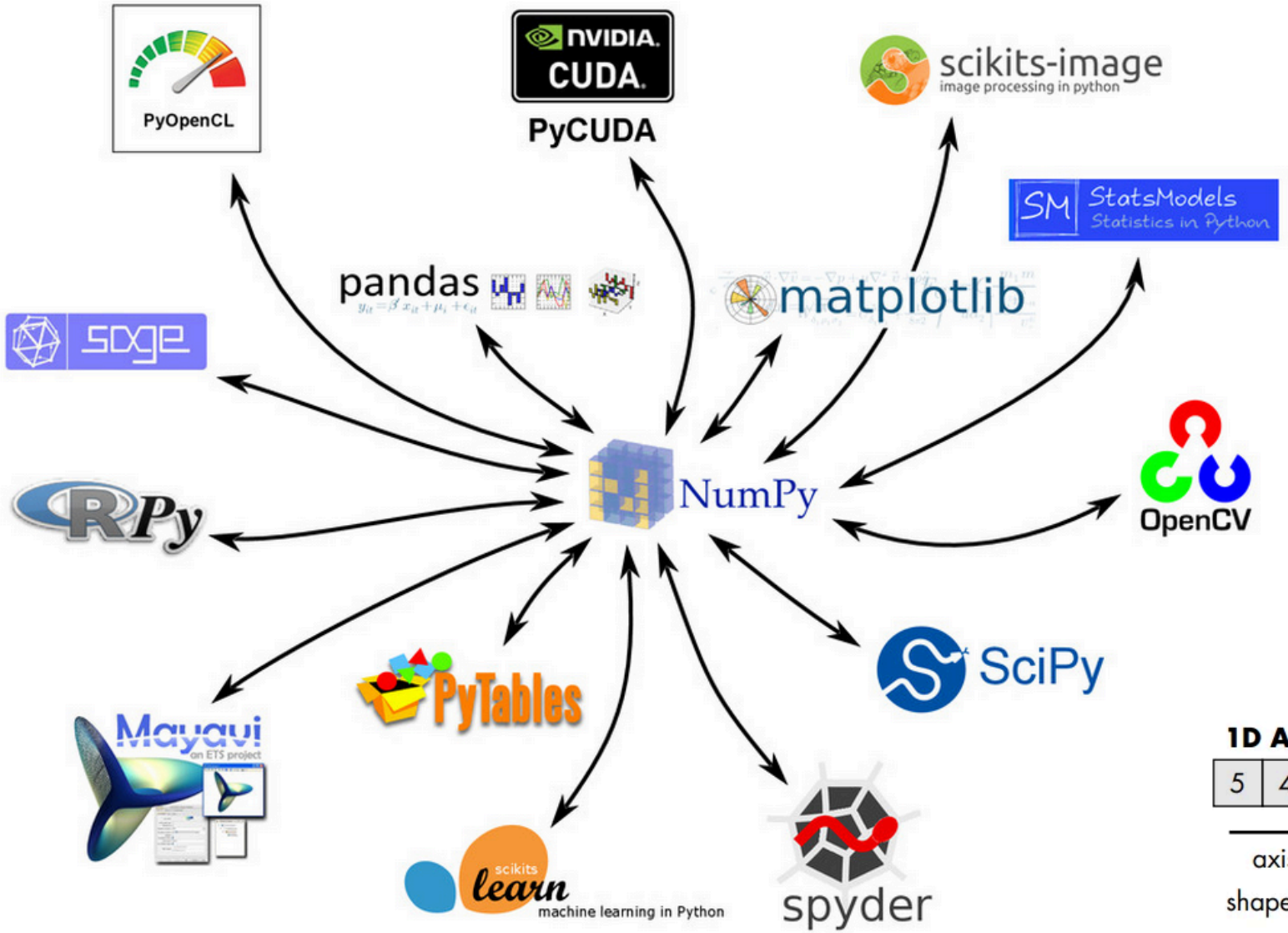
https://numpy.org/doc/2.1/user/numpy-for-matlab-users.html

What is NumPy?

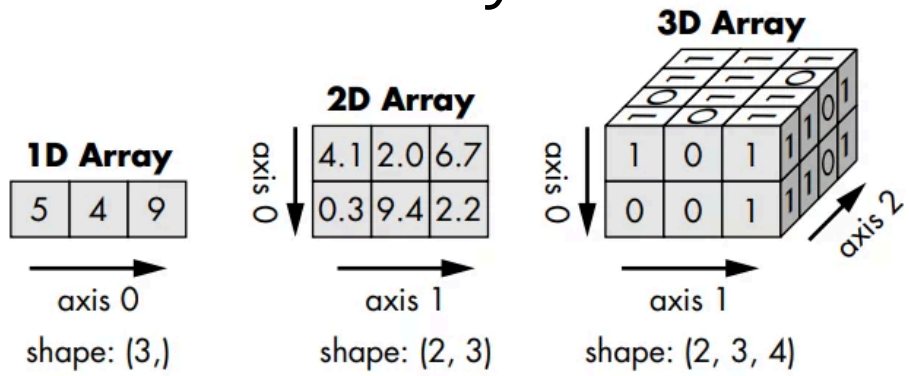
Numerical Python



The fundamental package for scientific computing in Python



nD-array

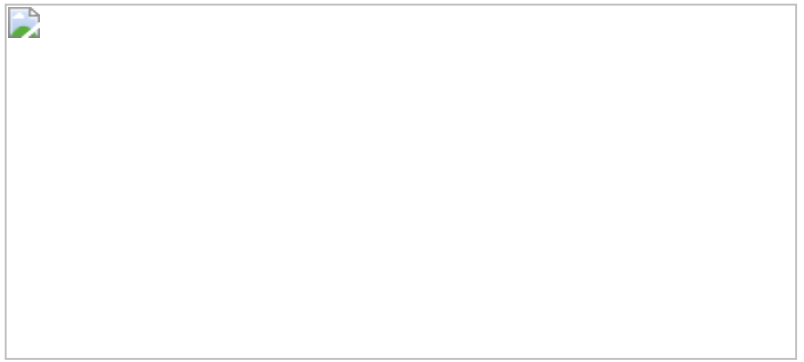


https://www.learndatasci.com/tutorials/applied-introduction-to-numpy-python-tutorial/

What is NumPy?

Numerical Python





The fundamental package for scientific computing in Python

Provides:

- Powerful N-dimensional array object (`ndarray`)
- Broadcasting functions
- Tools for integrating C/C++/Fortran code
- Linear algebra, Fourier transform and basic signal processing, random numbers

```
# The standard way to import NumPy
import numpy as np

# Check version
print(np.__version__)  # '2.3.0'
```

https://numpy.org/

What is NumPy?

Performance & Efficiency

Python Lists:

- *Flexible* - can store mixed types
- Heterogeneous storage
- Each element is a full Python object
- High memory overhead
- Slow for numerical operations

NumPy Arrays:

- *Homogeneous* - single data type
- Contiguous memory storage
- Fixed size at creation
- Low memory overhead
- **50-100x faster** for operations

```
# Example: Computing element-wise operations
python_list = list(range(1000000))
numpy_array = np.arange(1000000)

# Python list comprehension: ~100 ms
result_list = [x**2 for x in python_list]

# NumPy vectorized operation: ~1-2 ms
result_array = numpy_array**2  # 50-100x faster!
```

The ndarray object

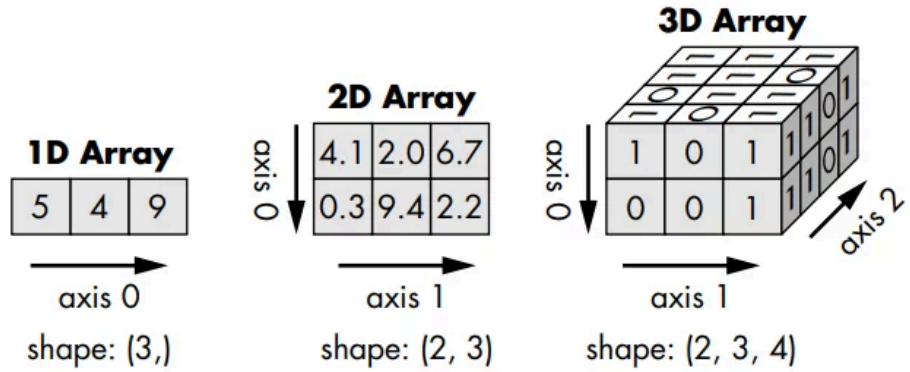
Vectors, matrices, and tensors

Key Properties:

- `ndim`: number of dimensions
- `shape`: size of each dimension
- `size`: total number of elements
- `dtype`: data type of elements

```python
# Create a 3D array (tensor)
tensor = np.array([
    [[1, 0, 1, 1],
     [0, 1, 1, 1],
     [1, 1, 0, 1]],
    [[0, 0, 1, 0],
     [0, 1, 0, 0],
     [1, 1, 0, 1]]
])

print(tensor.ndim)    # 3 (dimensions)
print(tensor.shape)   # (2, 3, 4)
print(tensor.size)    # 24 (total elements)
print(vector.dtype)   # int64 (or int32)
```



1D array = vector

2D array = matrix

3D+ array = tensor

The ndarray object

Creating arrays

```
# From a Python list
arr = np.array([1, 2, 3, 4, 5])

# From nested lists (2D)
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

# Zeros and ones
zeros = np.zeros((3, 4))
ones = np.ones((2, 3), dtype=int)

# Identity matrix
identity = np.eye(3)

# Empty array (uninitialized)
empty = np.empty((2, 2))

# Range of values
range_arr = np.arange(0, 10, 2)
# Output: [0, 2, 4, 6, 8]

# Linearly spaced values
linspace = np.linspace(0, 1, 5)
# Output: [0.  0.25  0.5  0.75  1. ]
```

```
# Random arrays
random_uniform = np.random.random((2, 3))
random_normal = np.random.randn(3, 3)
random_int = np.random.randint(0, 10, (2, 4))

# Full arrays (constant value)
full_arr = np.full((2, 3), 7)

# From a function
def func(x, y):
    return x + y

arr_func = np.fromfunction(func, (3, 3))

# Diagonal array
diag = np.diag([1, 2, 3, 4])

# Meshgrid (for 2D plotting)
x = np.linspace(0, 1, 3)
y = np.linspace(0, 1, 3)
X, Y = np.meshgrid(x, y)
```

The ndarray object

Array attributes and data types

Common Data Types (dtype):

- `int8, int16, int32, int64`
- `uint8, uint16, uint32, uint64`
- `float16, float32, float64`
- `complex64, complex128`
- `bool`
- `object` (Python objects)
- `string_, unicode_`

NumPy automatically chooses appropriate `dtype`, but you can specify it explicitly for memory efficiency

```
# Specify dtype explicitly
arr_int = np.array([1, 2, 3], dtype=np.int32)
arr_float = np.array([1, 2, 3], dtype=np.float64)

# Check dtype
print(arr_int.dtype)    # int32
print(arr_float.dtype)  # float64

# Type conversion (astype)
arr_converted = arr_int.astype(np.float32)

# Memory usage
print(arr_int.nbytes)    # bytes in memory

# Example: Neuroscience application
# EEG data: often float32 (saves memory)
eeg_data = np.random.randn(64, 1000)
eeg_data = eeg_data.astype(np.float32)

# Spike times: often int64 or float64
spike_times = np.array([12.5, 23.7, 45.2])
print(spike_times.dtype)  # float64
```

The ndarray object

Manipulating array dimensions

```python
# Create 1D array
x = np.arange(12)
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

# Reshape to 2D (3x4 matrix)
x_2d = x.reshape(3, 4)
# [[0,  1,  2,  3],
#  [4,  5,  6,  7],
#  [8,  9, 10, 11]]

# Reshape to 3D
x_3d = x.reshape(2, 3, 2)

# Auto-calculate one dimension with -1
x_auto = x.reshape(3, -1)  # 3x4
x_auto = x.reshape(-1, 4)  # 3x4

# Flatten to 1D
x_flat = x_2d.flatten()
x_ravel = x_2d.ravel()  # view (faster)

# Transpose
x_T = x_2d.T  # or x_2d.transpose() or np.transpo

# Add new axis
```

Shape Manipulation:

- `reshape()`: change shape / dimensions (must preserve size)
- `flatten()`: copy to 1D
- `ravel()`: view as 1D
- `transpose()` or `.T`: Swap axes
- `newaxis`: Add dimension

Stacking Arrays:

- `vstack()`: stack vertically
- `hstack()`: stack horizontally
- `concatenate()`: join along axis

```python
# Stacking examples
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

v_stack = np.vstack((a, b))  # (4, 2)
```

The ndarray object

Indexing and Slicing

```
# 1D array indexing
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

x[2]        # 2 (single element)
x[-1]       # 9 (last element)
x[2:5]      # [2, 3, 4] (slice)
x[::2]      # [0, 2, 4, 6, 8] (step)
x[::-1]     # [9, 8, 7, ..., 0] (reverse)

# 2D array indexing
y = np.array([[0, 1, 2, 3],
              [4, 5, 6, 7],
              [8, 9, 10, 11]])

y[1, 2]         # 6 (single element)
y[1]            # [4, 5, 6, 7] (entire row)
y[:, 2]         # [2, 6, 10] (entire column)
y[0:2, 1:3]     # [[1, 2], [5, 6]] (subarray)

# Negative indexing
y[-1, -1]       # 11 (last element)
y[-2:]          # last 2 rows

# Multiple indices at once
y[[0, 2], [1, 3]]  # [1, 11]
```

Key Points:

- **Zero-based indexing**
- Slices are **views**, not copies
- Use `[start:stop:step]` syntax
- Negative indices count from end
- `:` alone = all elements in dimension

```
# EEG data: (64 channels, 1000 time points)
eeg = np.random.randn(64, 1000)

# Get channel 5
ch5 = eeg[4, :]  # or simply eeg[4]

# Get first 100 samples from all channels
first_100 = eeg[:, :100]

# Get channels 10-20, samples 200-300
```

The ndarray object

Advanced selection techniques

```
# Boolean indexing (masking)
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Create boolean mask
mask = x > 5
# [False, False, False, False, False,
#  False, True, True, True, True]

# Use mask to select elements
x[mask]  # [6, 7, 8, 9]

# Or in one line
x[x > 5]  # [6, 7, 8, 9]

# Complex conditions
x[(x > 2) & (x < 7)]  # [3, 4, 5, 6]
x[(x < 3) | (x > 7)]  # [0, 1, 2, 8, 9]

# Find indices where condition is True
indices = np.where(x > 5)
# (array([6, 7, 8, 9]),)

# Modify elements based on condition
x[x > 5] = 0  # Set all values > 5 to 0

# Count True values
```

```
# Fancy indexing (array indexing)
x = np.array([10, 20, 30, 40, 50])

# Use array of indices
indices = np.array([0, 2, 4])
x[indices]  # [10, 30, 50]

# Can use in any order
indices = np.array([4, 0, 2, 2])
x[indices]  # [50, 10, 30, 30]

# 2D fancy indexing
y = np.arange(12).reshape(3, 4)
row_idx = np.array([0, 1, 2])
col_idx = np.array([2, 3, 1])
y[row_idx, col_idx]  # [2, 7, 9]
```

The ndarray object

```
# Element-wise operations
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

# Arithmetic (element-wise)
a + b       # [11, 22, 33, 44]
a - b       # [-9, -18, -27, -36]
a * b       # [10, 40, 90, 160]
a / b       # [0.1, 0.1, 0.1, 0.1]
a ** 2      # [1, 4, 9, 16]

# Comparison (returns boolean array)
a > 2       # [False, False, True, True]
a == b      # [False, False, False, False]

# Universal functions (ufuncs)
np.sin(a)
np.exp(a)
np.log(a)
np.sqrt(a)
np.abs(a)

# Aggregate functions
a.sum()         # 10
a.mean()        # 2.5
a.std()         # standard deviation
a.min()         # 1
a.max()         # 4
a.argmin()      # 0 (index of min)
```

Array Operations and Broadcasting

Broadcasting:

- NumPy automatically extends smaller arrays to match larger ones during operations

Rules:

- Arrays with different dimensions can be combined
- Dimensions are compared from right to left
- Compatible if dimensions are equal or one is 1

```
# Broadcasting examples
a = np.array([[1, 2, 3],
              [4, 5, 6]])  # (2, 3)

# Add scalar to array (broadcasts to all elements)
a + 10  # [[11, 12, 13], [14, 15, 16]]

# Add 1D array to 2D array
b = np.array([10, 20, 30])  # (3,)
a + b  # [[11, 22, 33], [14, 25, 36]]

# Add column vector to 2D array
c = np.array([[10], [20]])  # (2, 1)
a + c  # [[11, 12, 13], [24, 25, 26]]

# Normalize rows (subtract mean of each row)
a - a.mean(axis=1, keepdims=True)
```

The ndarray object

Array Manipulation Functions

```
# Sorting
x = np.array([3, 1, 4, 1, 5, 9, 2, 6])
np.sort(x)          # [1, 1, 2, 3, 4, 5, 6, 9]
np.argsort(x)       # indices that would sort
x.sort()            # in-place sort

# Unique values
np.unique(x)        # [1, 2, 3, 4, 5, 6, 9]
np.unique(x, return_counts=True)

# Concatenate
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.concatenate((a, b))  # [1, 2, 3, 4, 5, 6]

# Repeat and tile
np.repeat(a, 3)     # [1, 1, 1, 2, 2, 2, 3, 3, 3]
np.tile(a, 3)       # [1, 2, 3, 1, 2, 3, 1, 2, 3]

# Splitting
x = np.arange(9)
np.split(x, 3)      # [array([0,1,2]), ...]

# Difference between consecutive elements
np.diff(x)          # [1, 1, 1, 1, 1, 1, 1, 1]

# Cumulative sum
```

```
# Linear algebra operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = A @ B           # or np.dot(A, B)
C = np.matmul(A, B)

# Element-wise multiplication
A * B
```

```
# Statistical functions
data = np.random.randn(100)

# Basic statistics
np.mean(data)
np.median(data)
np.std(data)        # standard deviation
np.var(data)        # variance
np.percentile(data, 95)

# Along specific axis
matrix = np.random.randn(10, 5)
matrix.mean(axis=0)  # mean of each column
matrix.mean(axis=1)  # mean of each row

# Correlation
```

```
help(np.linalg)
help(np.random)
```

The ndarray object

Aliasing vs Copying

```
# Assignment = ALIAS (same data!)
x = np.array([1, 2, 3, 4])
y = x                # y points to same data

y[0] = 99
print(x)             # [99, 2, 3, 4] - CHANGED!

# Slicing = VIEW (still same data!)
z = x[1:3]           # view, not copy
z[0] = 88
print(x)             # [99, 88, 3, 4] - CHANGED!

# Explicit copy = INDEPENDENT
w = x.copy()         # new independent array
w[0] = 77
print(x)             # [99, 88, 3, 4] - unchange
print(w)             # [77, 88, 3, 4]

# Check if view or copy
x.base is None       # True = owns data
```

Rule of thumb: When in doubt, use `.copy()` to avoid unexpected side effects!

Views (same data):

- Assignment: `y = x`
- Slicing: `x[1:5]`
- Reshape: `x.reshape()`
- Transpose: `x.T`
- Ravel: `x.ravel()`

Copies (new data):

- Explicit: `x.copy()`
- Flatten: `x.flatten()`
- Fancy indexing: `x[[1,2,3]]`
- Boolean indexing: `x[x > 5]`

CRITICAL CONCEPT: NumPy arrays can be **views** (references to same data) or **copies** (independent data)

What Else Can NumPy Do?

```
# Linear Algebra (numpy.linalg)
import numpy as np

# Matrix operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Determinant
det_A = np.linalg.det(A)

# Matrix inverse
A_inv = np.linalg.inv(A)

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)

# Solve linear system Ax = b
b = np.array([1, 2])
x = np.linalg.solve(A, b)

# Singular Value Decomposition (SVD)
U, s, Vt = np.linalg.svd(A)

# Matrix rank
rank = np.linalg.matrix_rank(A)

# Norm of a vector/matrix
norm = np.linalg.norm(A)
```

Linear algebra and signal processing

```
# Basic Signal Processing
# Convolution (filtering, smoothing)
signal = np.random.randn(1000)
kernel = np.ones(10) / 10  # moving average
smoothed = np.convolve(signal, kernel, mode='same')

# Correlation
signal1 = np.random.randn(100)
signal2 = np.random.randn(100)
correlation = np.correlate(signal1, signal2, mode='full')

# Correlation coefficient matrix
data_matrix = np.random.randn(5, 100)  # 5 channels
corr_matrix = np.corrcoef(data_matrix)

# FFT (Fast Fourier Transform)
# Note: use scipy.fft for more advanced features
fft_result = np.fft.fft(signal)
frequencies = np.fft.fftfreq(len(signal), d=0.001)
power_spectrum = np.abs(fft_result)**2

# Neuroscience example: connectivity analysis
eeg_channels = np.random.randn(64, 1000)
connectivity = np.corrcoef(eeg_channels)
```

What is SciPy?

Scientific Python library

SciPy = Scientific Python

Open-source library built on NumPy that provides:

- High-level scientific algorithms
- Mathematical functions and routines
- Optimized implementations from Fortran/C libraries
- Tools for common scientific tasks



SciPy builds on NumPy arrays and provides specialized functionality for scientific and technical computing

```
# Import SciPy
import scipy
from scipy import stats, signal, optimize, integra

# Check version
print(scipy.__version__)  # '1.16.3'
```

What is SciPy?

SciPy is organized into submodules, each focused on a specific domain:

Module	Functionality
`scipy.stats`	Statistical functions, distributions, hypothesis tests
`scipy.signal`	Signal processing: filtering, convolution, spectral analysis
`scipy.optimize`	Optimization algorithms: minimization, curve fitting, root finding
`scipy.integrate`	Numerical integration: ODEs, quadrature
`scipy.interpolate`	Interpolation techniques: 1D, 2D, splines
`scipy.linalg`	Linear algebra: matrix operations, decompositions
`scipy.fft`	Fast Fourier Transform algorithms
`scipy.ndimage`	N-dimensional image processing
`scipy.spatial`	Spatial data structures and algorithms
`scipy.sparse`	Sparse matrices and associated routines

What is SciPy?

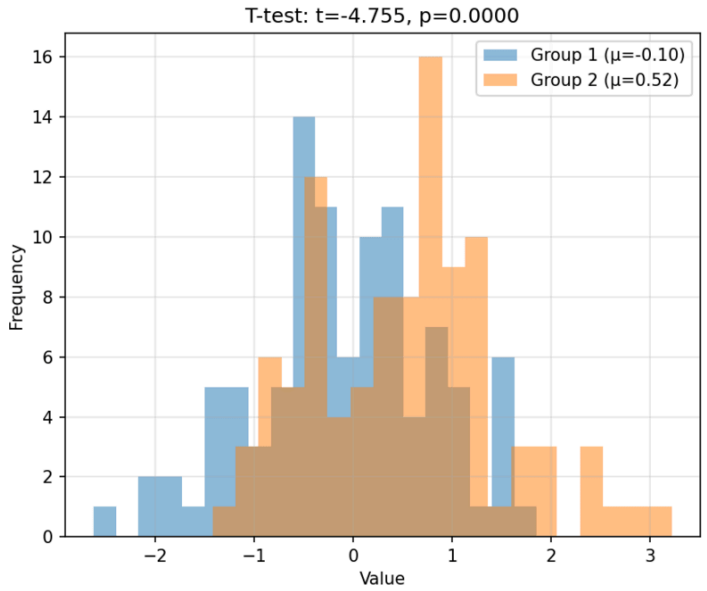
Practical Examples

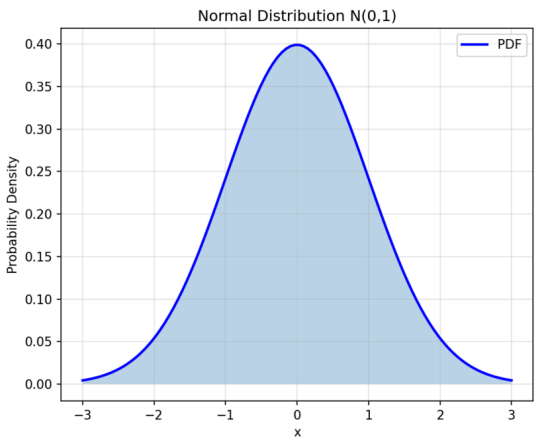
```python
# Statistics (scipy.stats)
from scipy import stats

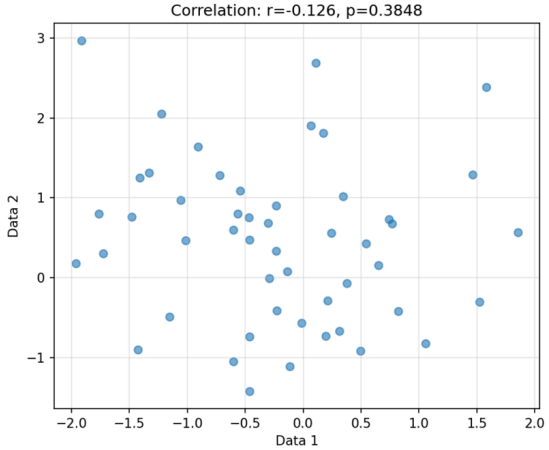
# T-test
data1 = np.random.randn(100)
data2 = np.random.randn(100) + 0.5
t_stat, p_value = stats.ttest_ind(data1, data2)

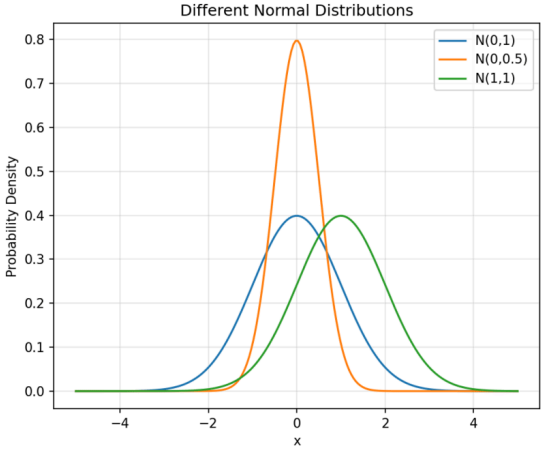
# Normal distribution
x = np.linspace(-3, 3, 100)
pdf = stats.norm.pdf(x, loc=0, scale=1)

# Correlation
```









Note: plots generated with `matplotlib`

What is SciPy?

Practical Examples

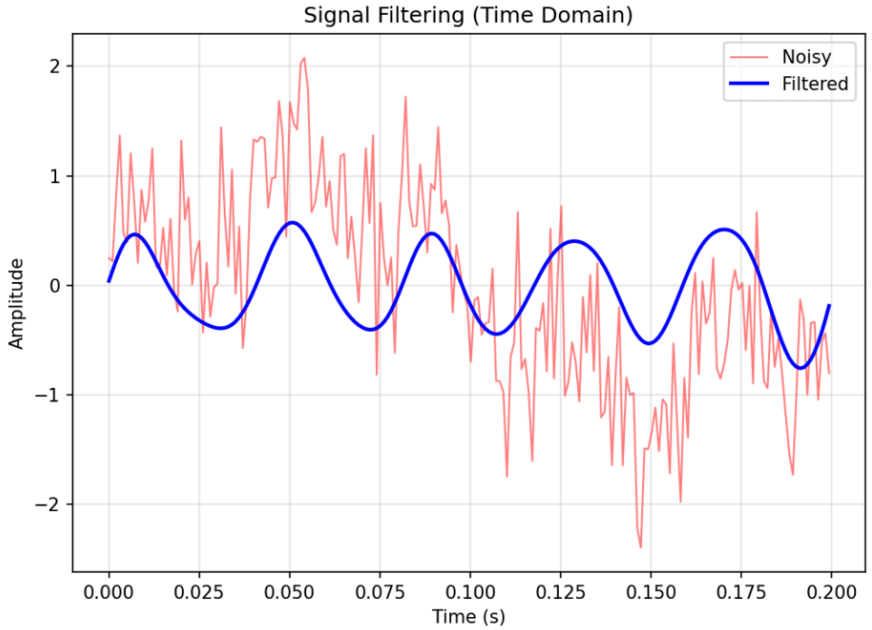
```
# Signal processing (scipy.signal)
from scipy import signal

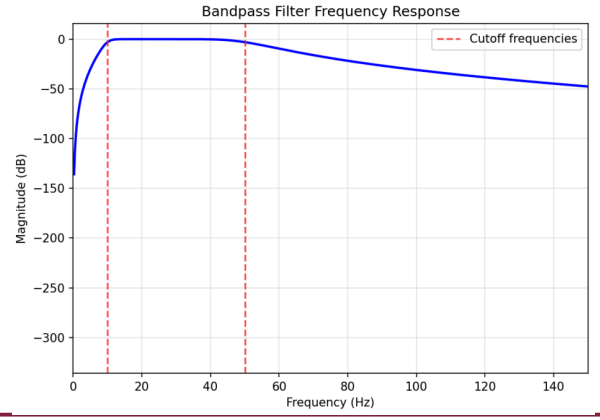
# Design a bandpass filter
fs = 1000  # sampling frequency

# Create signal: 5 Hz + 25 Hz + 100 Hz + noise
signal_clean = (np.sin(2*np.pi*5*t) +
                0.5*np.sin(2*np.pi*25*t) +
                0.3*np.sin(2*np.pi*100*t))
noise = 0.5*np.random.randn(len(t))
signal_noisy = signal_clean + noise

# Design a bandpass filter
lowcut = 10
highcut = 50
b, a = signal.butter(4, [lowcut, highcut],
                     btype='band', fs=fs)

# Apply filter
filtered = signal.filtfilt(b, a, signal_noisy)
```





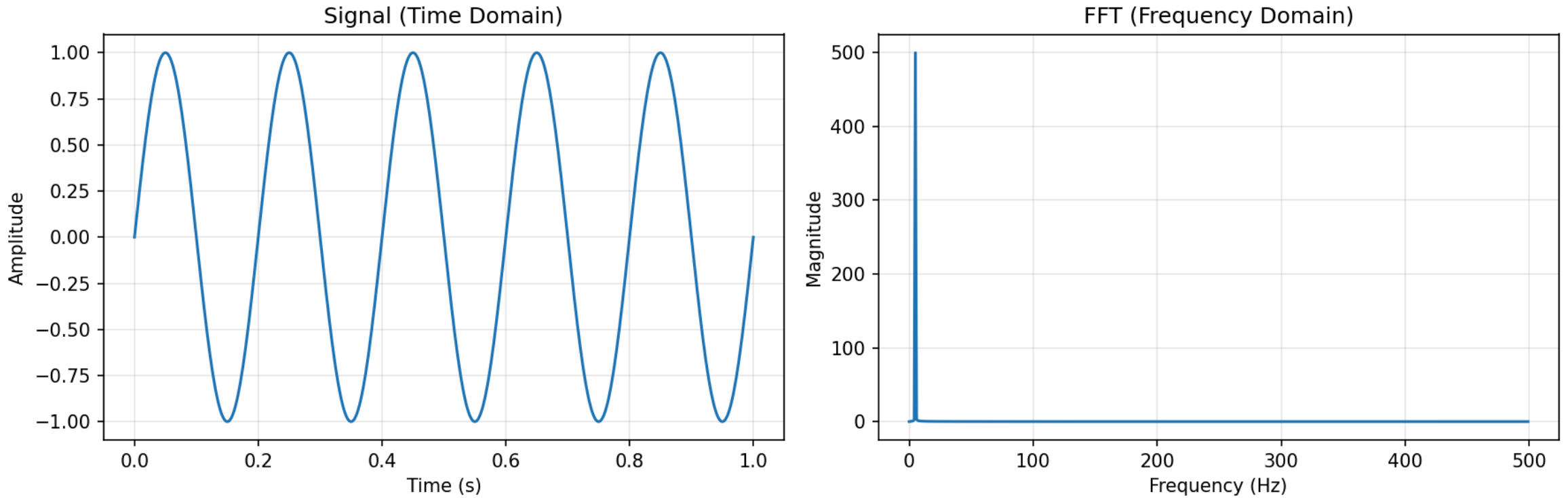
Note: plots generated with `matplotlib`

What is SciPy?

Practical Examples

```
# FFT (scipy.fft)
from scipy import fft

signal_data = np.sin(2*np.pi*5*np.linspace(0,1,1000))
spectrum = fft.fft(signal_data)
frequencies = fft.fftfreq(len(signal_data), 1/1000)
```



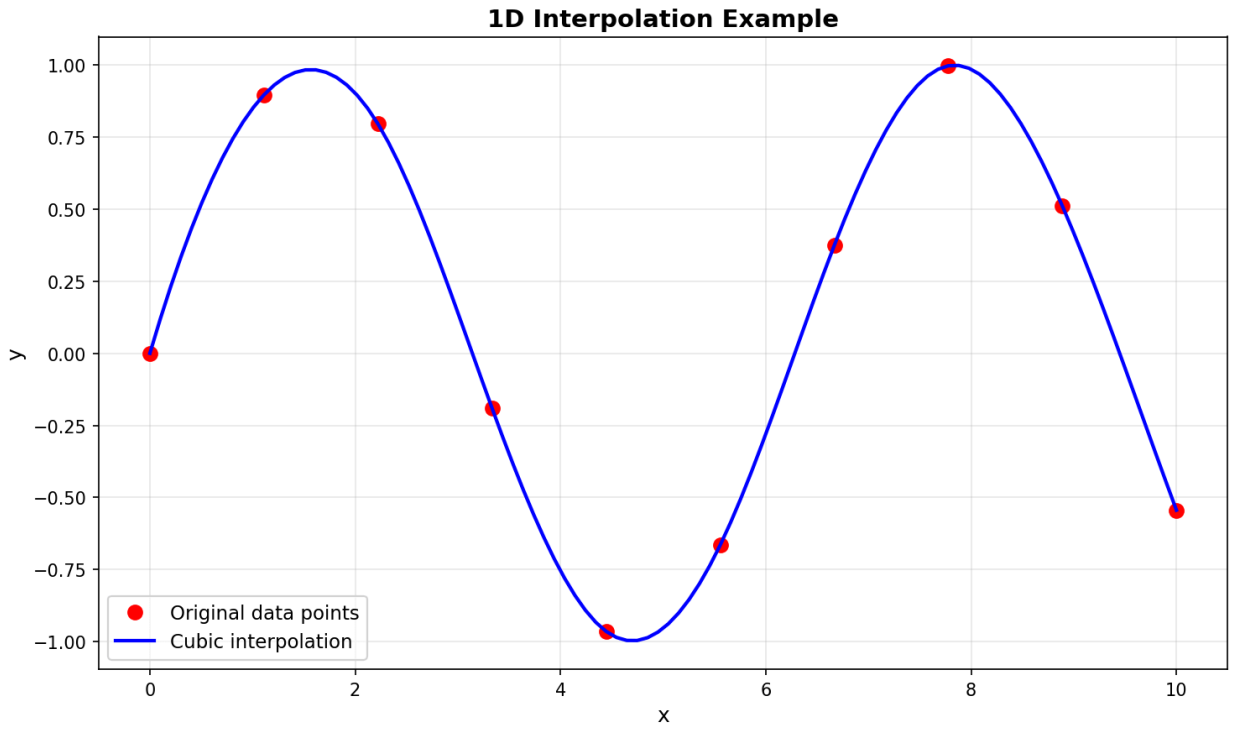
Note: plots generated with `matplotlib`

What is SciPy?

Practical Examples

```python
# Interpolation (scipy.interpolate)
from scipy import interpolate

# 1D interpolation
x = np.linspace(0, 10, 10)
y = np.sin(x)
f = interpolate.interp1d(x, y, kind='cubic')
x_new = np.linspace(0, 10, 100)
y_new = f(x_new)
```



Note: plots generated with `matplotlib`

What is SciPy?

```python
# Optimization (scipy.optimize)
from scipy import optimize

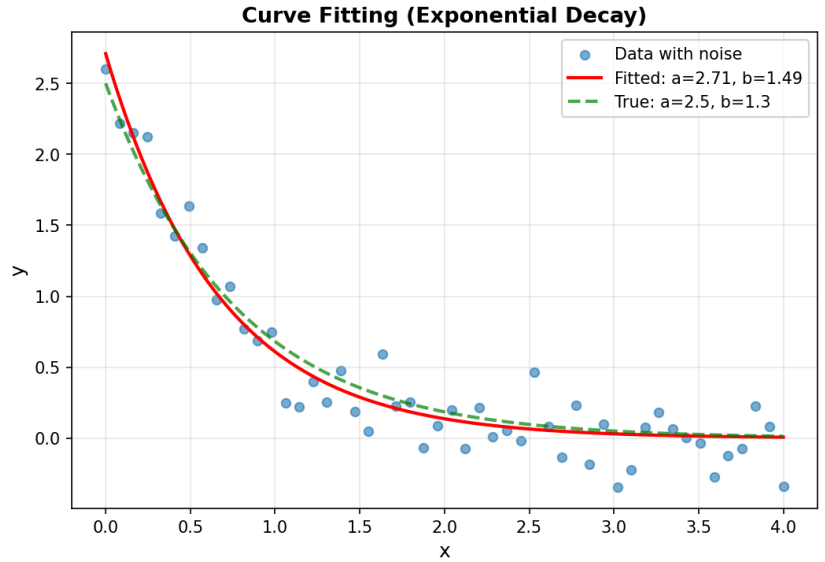
# Curve fitting
def func(x, a, b):
    return a * np.exp(-b * x)

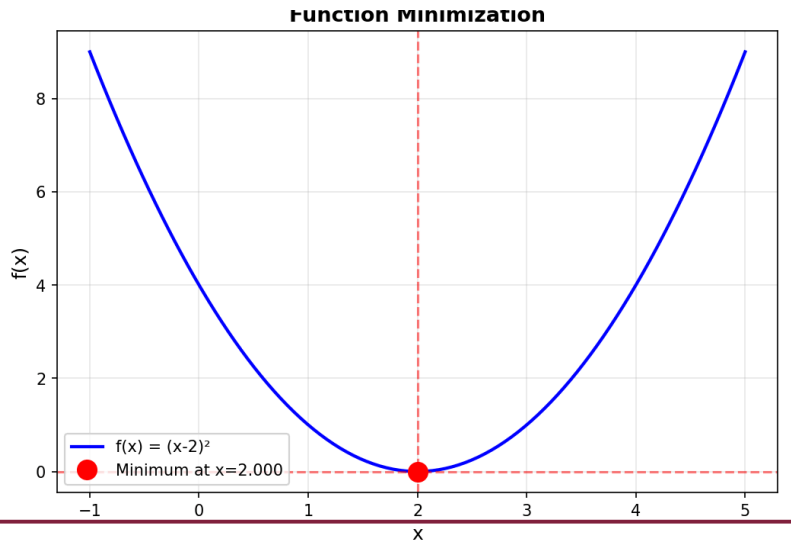
x_data = np.linspace(0, 4, 50)
y_data = func(x_data, 2.5, 1.3) + 0.2*np.random.randn(50)
params, covariance = optimize.curve_fit(func, x_data, y_data

# Find minimum of function
def parabola(x):
    return (x-2)**2

result = optimize.minimize(parabola, x0=0)
```

Practical Examples





Note: plots generated with `matplotlib`

SciPy + NumPy

Understanding the relationship



```
# Typical imports
import numpy as np
from scipy import signal, stats
```



Foundation

Basic array operations and data structures

Core functionality

Array creation, manipulation, basic math

Fast operations

Element-wise operations, broadcasting

Always needed

Required for scientific Python work

Examples

`np.array()`, `np.mean()`, `np.sum()`

Extension

Advanced scientific algorithms built on NumPy

Specialized tools

Statistics, optimization, signal processing

Complex algorithms

FFT, integration, interpolation, filtering

Task-specific

Import only the modules you need

Examples

`stats.ttest()`, `signal.butter()`, `optimize.minimize()`

Key relationship: NumPy provides the array infrastructure, SciPy provides the scientific algorithms that operate on those arrays. You typically import both for scientific work.

Data Visualization

Essential for scientific research

Visualization is crucial for:

- **Exploring data**
  Discover patterns, outliers, relationships
- **Interpreting results**
  Understand what your analysis reveals
- **Comparing experiments**
  Theory vs observation, conditions vs controls
- **Communicating findings**
  Papers, presentations, lab meetings
- **Quality control**
  Spot artifacts, verify preprocessing



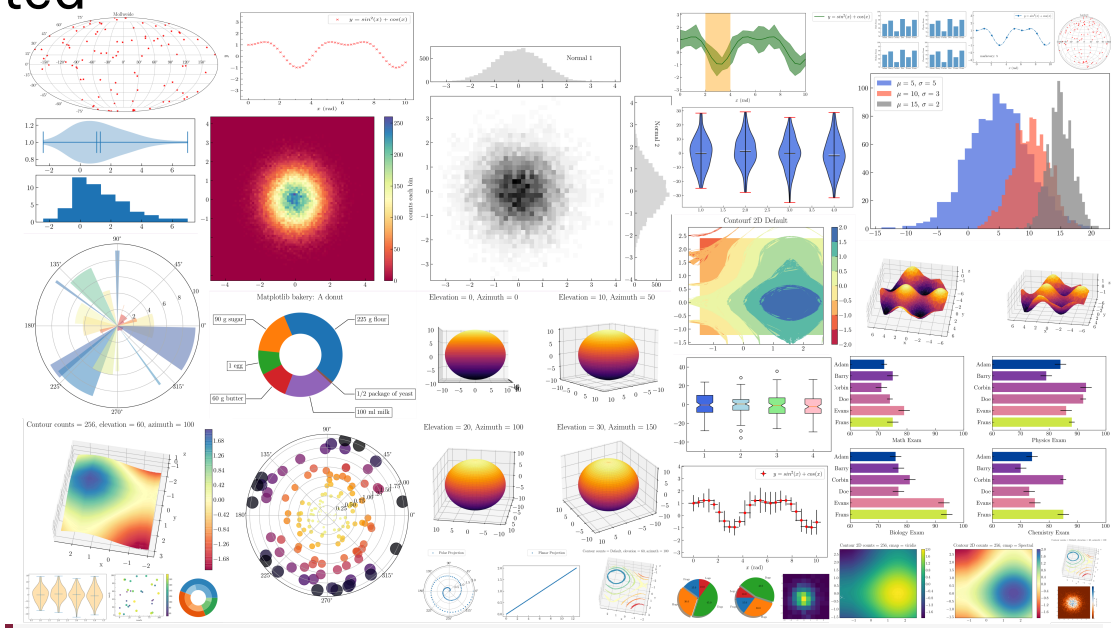
"A picture is worth a thousand words, but a good scientific plot is worth a thousand data points."

What is matplotlib?

Python's plotting library

MatPlotLib = Mathematical Plotting Library

Comprehensive library for creating static, animated and interactive visualizations in Python



Key Features:

- Publication-quality figures
- Works with NumPy arrays
- MATLAB-like interface (pyplot)
- Highly customizable
- Multiple output formats (PNG, PDF, SVG)
- Interactive and non-interactive modes

```
# Standard import convention
import matplotlib.pyplot as plt
import numpy as np

# Check version
```

https://matplotlib.org/stable/

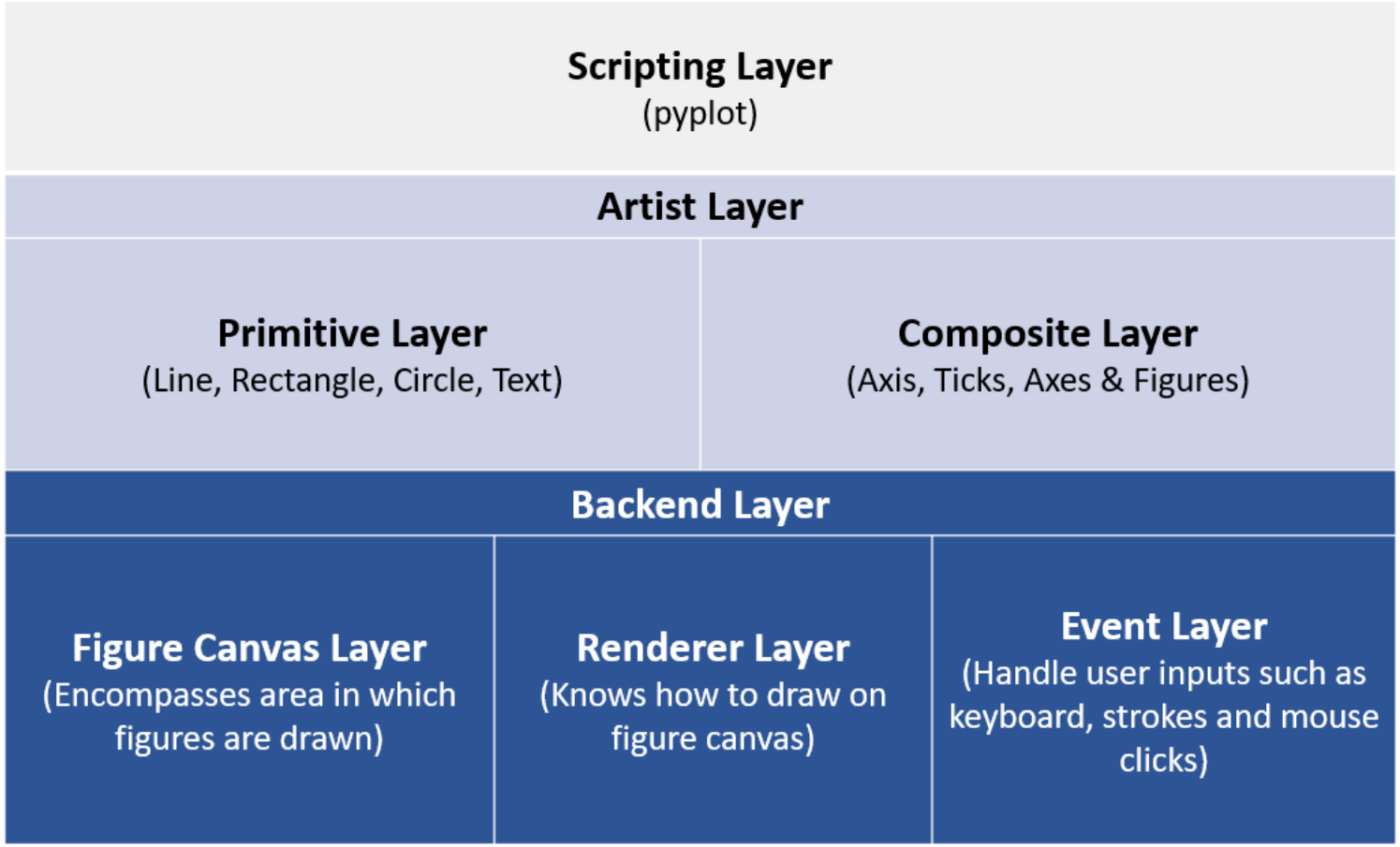
The most widely used plotting library in scientific Python

What is matplotlib?

Architecture

Hierarchy of Objects:

- **Figure**
  - The entire window or page
  - Contains one or more Axes
- **Axes**
  - The plotting area (NOT axis!)
  - Contains the actual plot
- **Axis**
  - The x-axis and y-axis
  - Controls ticks, labels, limits
- **Artists**
  - Everything you see on the figure
  - Lines, text, patches, etc.



Two interfaces: pyplot (MATLAB-style, stateful, easier for beginners) and **object-oriented** (explicit, more control, better for complex plots)

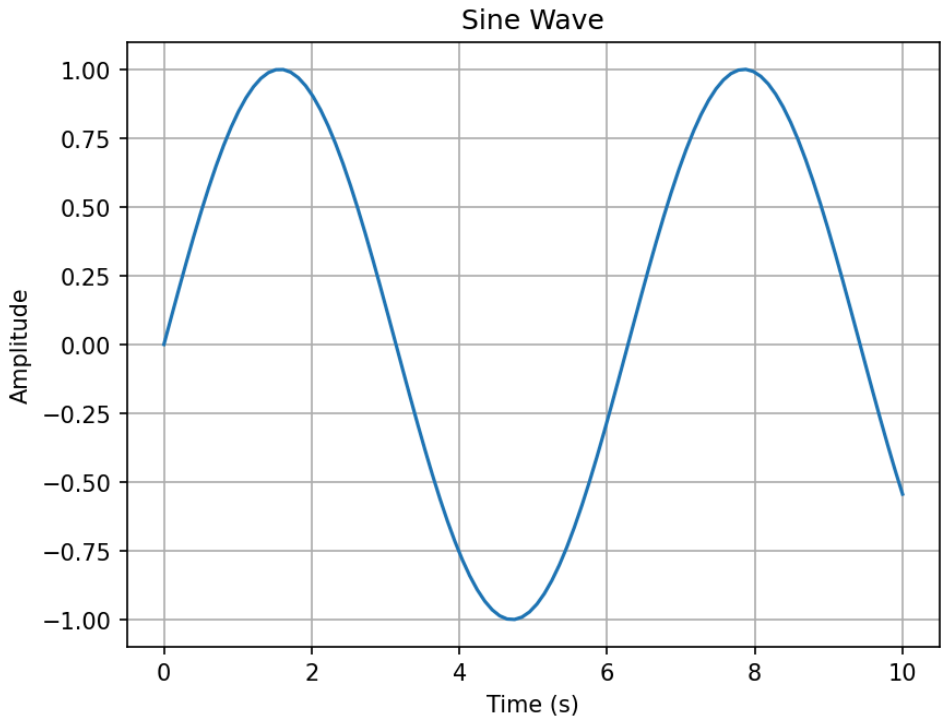
Basic Plotting with pyplot

Creating your first plots

```python
# Simple line plot
import matplotlib.pyplot as plt
import numpy as np

# Generate data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Create plot
plt.plot(x, y)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Sine Wave')
plt.grid(True)
plt.show()
```



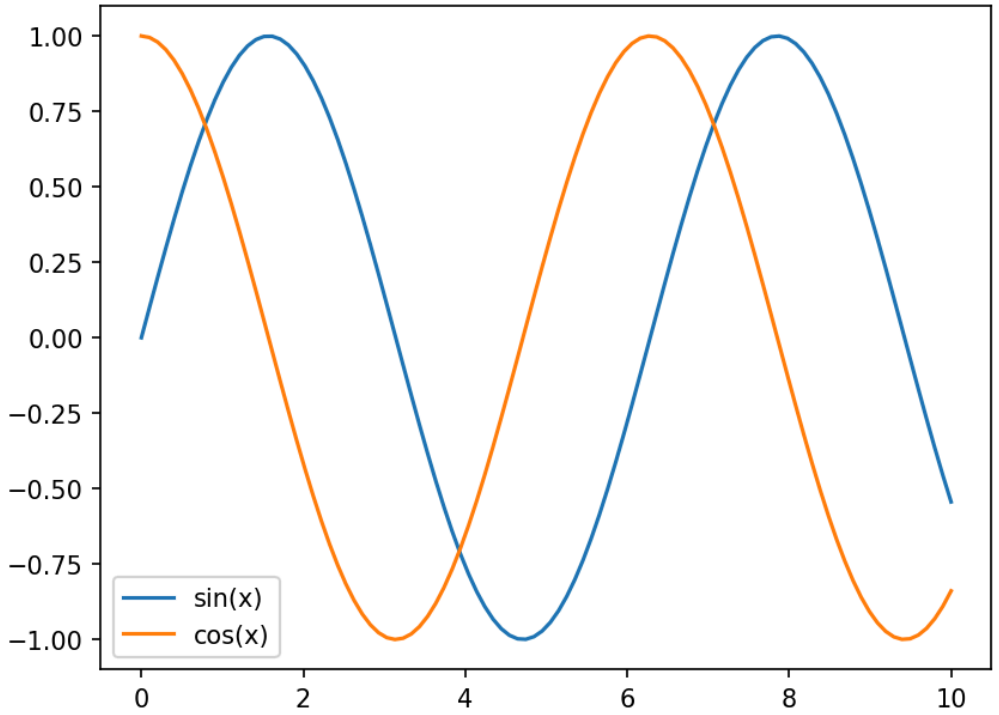
Basic Plotting with pyplot

Creating your first plots

```python
# Simple line plot
import matplotlib.pyplot as plt
import numpy as np

# Generate data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Multiple lines
y2 = np.cos(x)
plt.plot(x, y, label='sin(x)')
plt.plot(x, y2, label='cos(x)')
plt.legend()
plt.show()
```



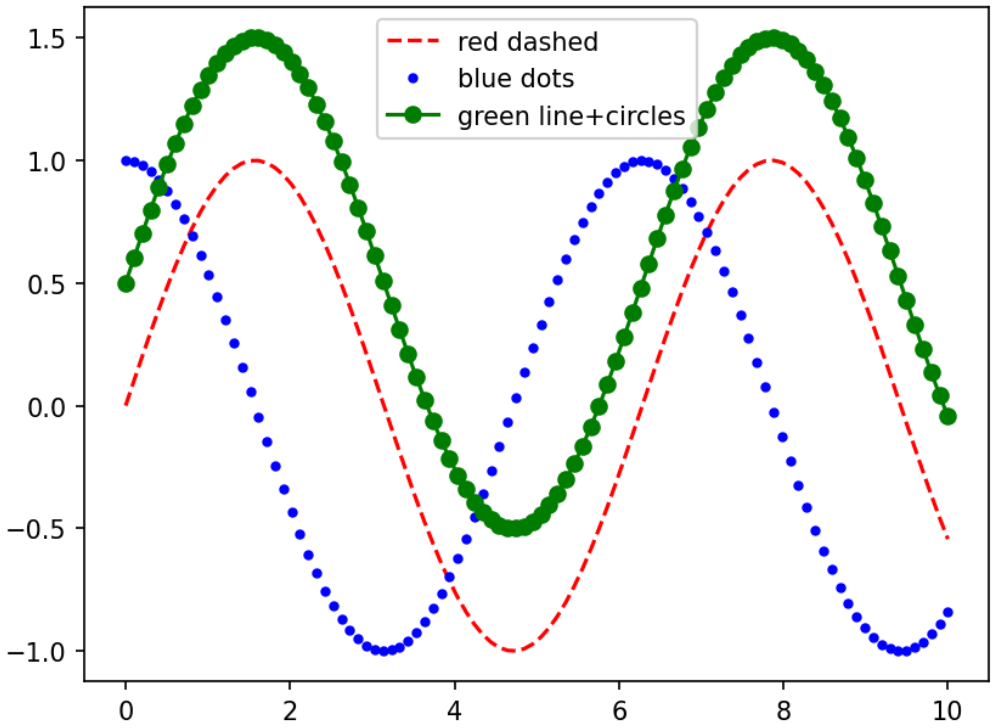
Basic Plotting with pyplot

Creating your first plots

```
# Simple line plot
import matplotlib.pyplot as plt
import numpy as np

# Generate data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Line styles and markers
plt.plot(x, y, 'r--', label='red dashed')
plt.plot(x, y2, 'b.', label='blue dots')
plt.plot(x, y+0.5, 'go-', label='green line+circles'
plt.legend()
plt.savefig(os.path.join(output_dir, 'fig3.png'), dp
plt.show()
```



Basic Plotting with pyplot

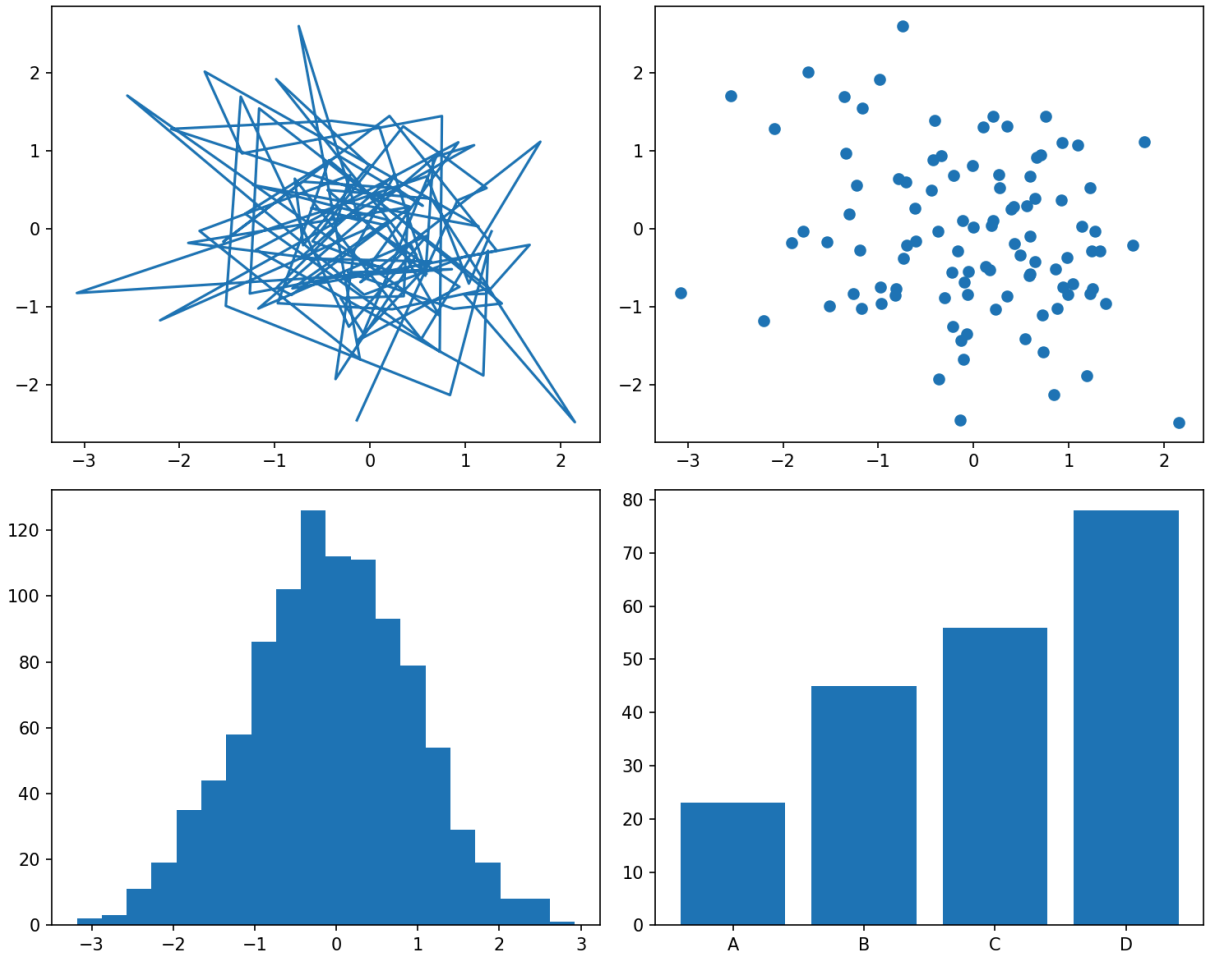
```
# Scatter plot
x = np.random.randn(100)
y = np.random.randn(100)
plt.scatter(x, y, alpha=0.5)
plt.xlabel('Variable 1')
plt.ylabel('Variable 2')
plt.show()

# Bar plot
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]
plt.bar(categories, values)
plt.ylabel('Count')
plt.show()

# Histogram
data = np.random.randn(1000)
plt.hist(data, bins=30, edgecolor='black')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()

# Subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].plot(x, y)
axes[0, 1].scatter(x, y)
axes[1, 0].hist(data, bins=20)
axes[1, 1].bar(categories, values)
plt.tight_layout()
```

Creating your first plots



Customizing Plots

Making publication-quality figures

```
# Colors
x = np.li
y = np.si

# Color
# Or RGB
plt.plot

# Line st
plt.plot

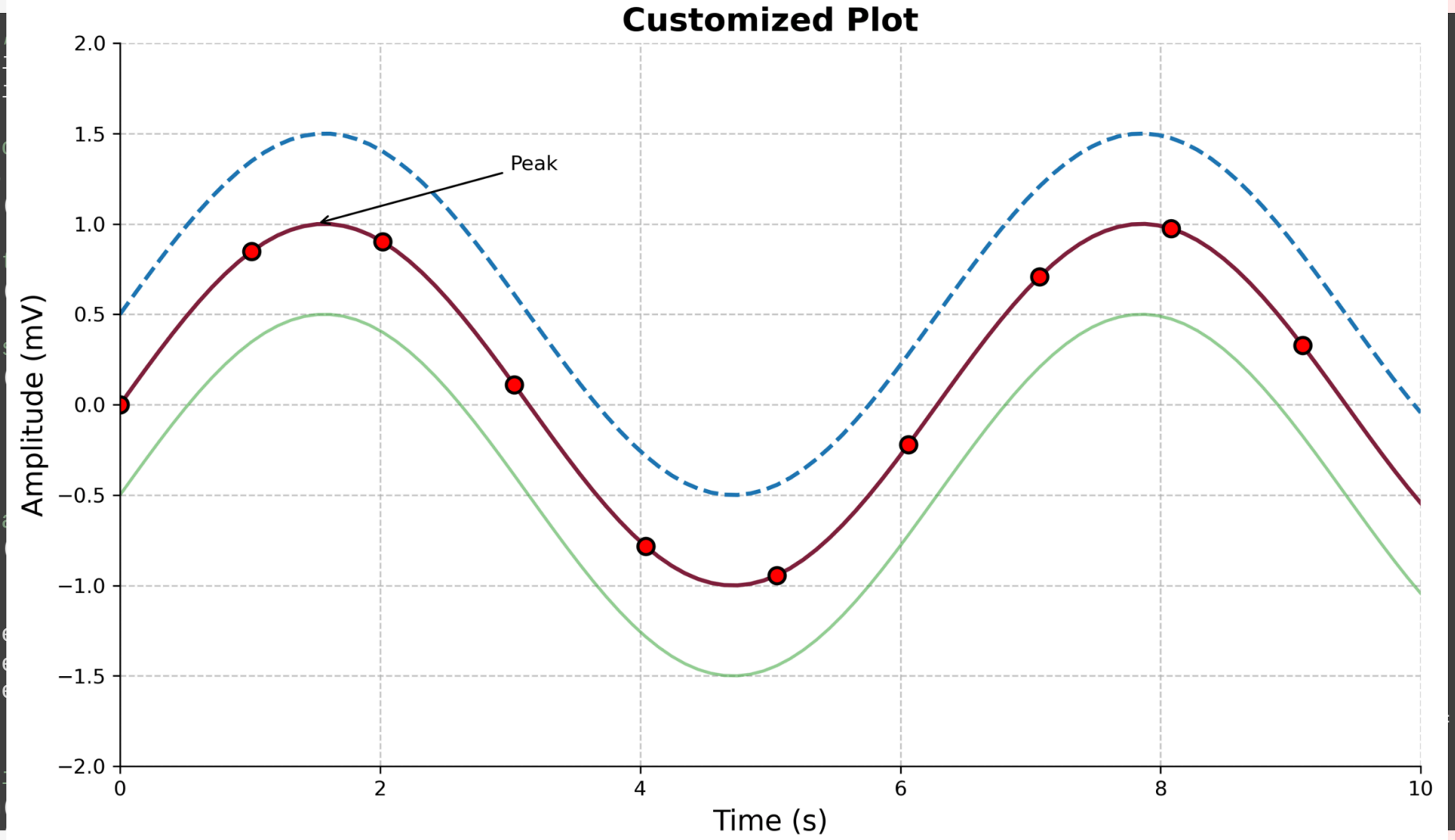
# Markers
plt.plot



# Transpa
plt.plot

# Labels
plt.xlabe
plt.ylabe
plt.title


# Axis li
plt.xlim
```




```
'tight')
```

Plotting Histograms

Visualizing distributions

```python
# Generate sample data
np.random.seed(42)
data = np.random.normal(15, 10, 1000)

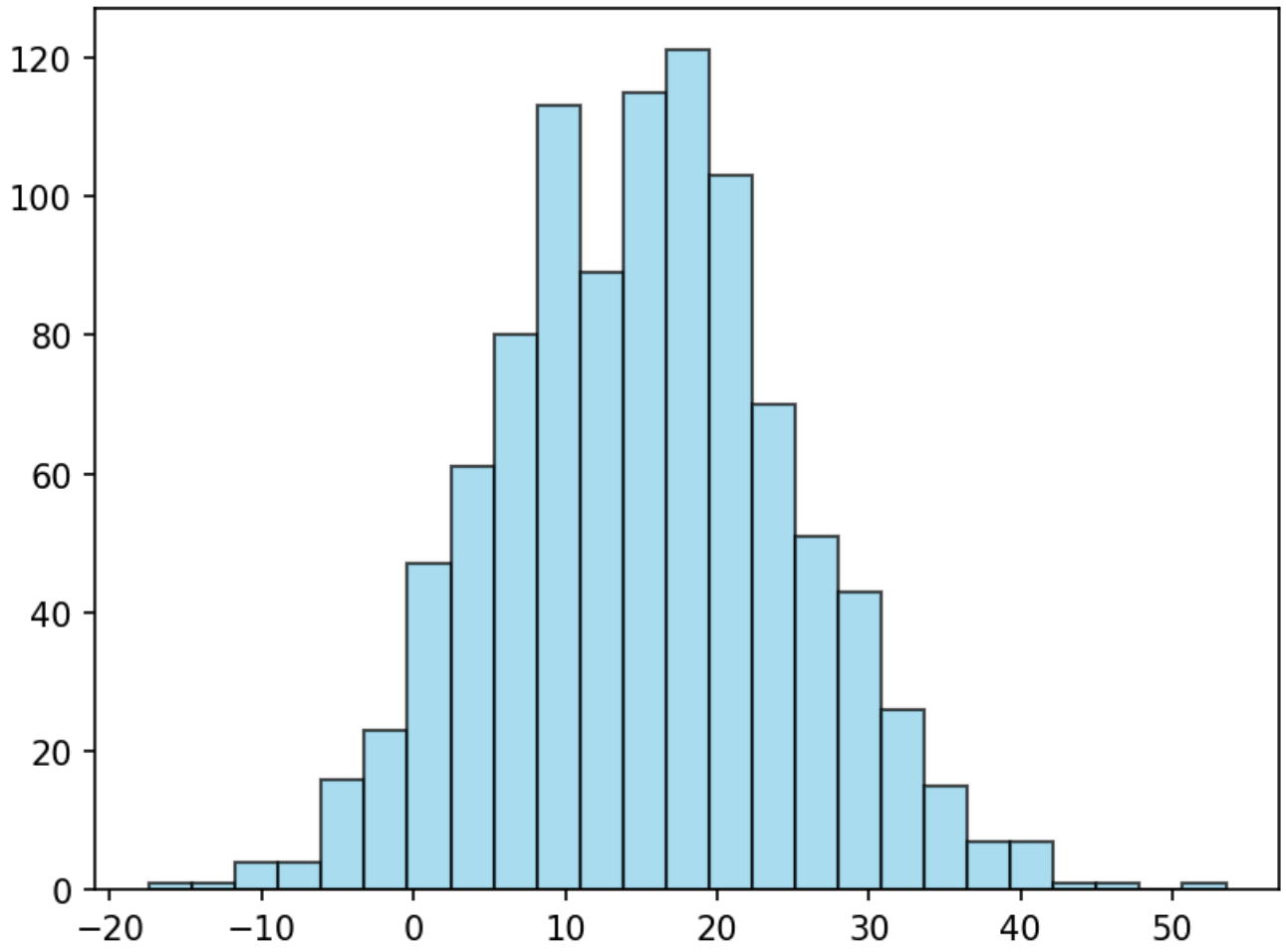
# Basic histogram
plt.hist(data)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Basic Histogram')
plt.show()

# Specify number of bins
plt.hist(data, bins=30)
plt.show()

# Specify bin edges
bins = np.arange(0, 50, 2)
plt.hist(data, bins=bins)
plt.show()

# Normalized histogram (density)
plt.hist(data, bins=30, density=True)
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.show()

# Customize appearance
plt.hist(data, bins=25,
         edgecolor='black',
         facecolor='skyblue',
         alpha=0.7)
```



Plotting Histograms

Visualizing distributions

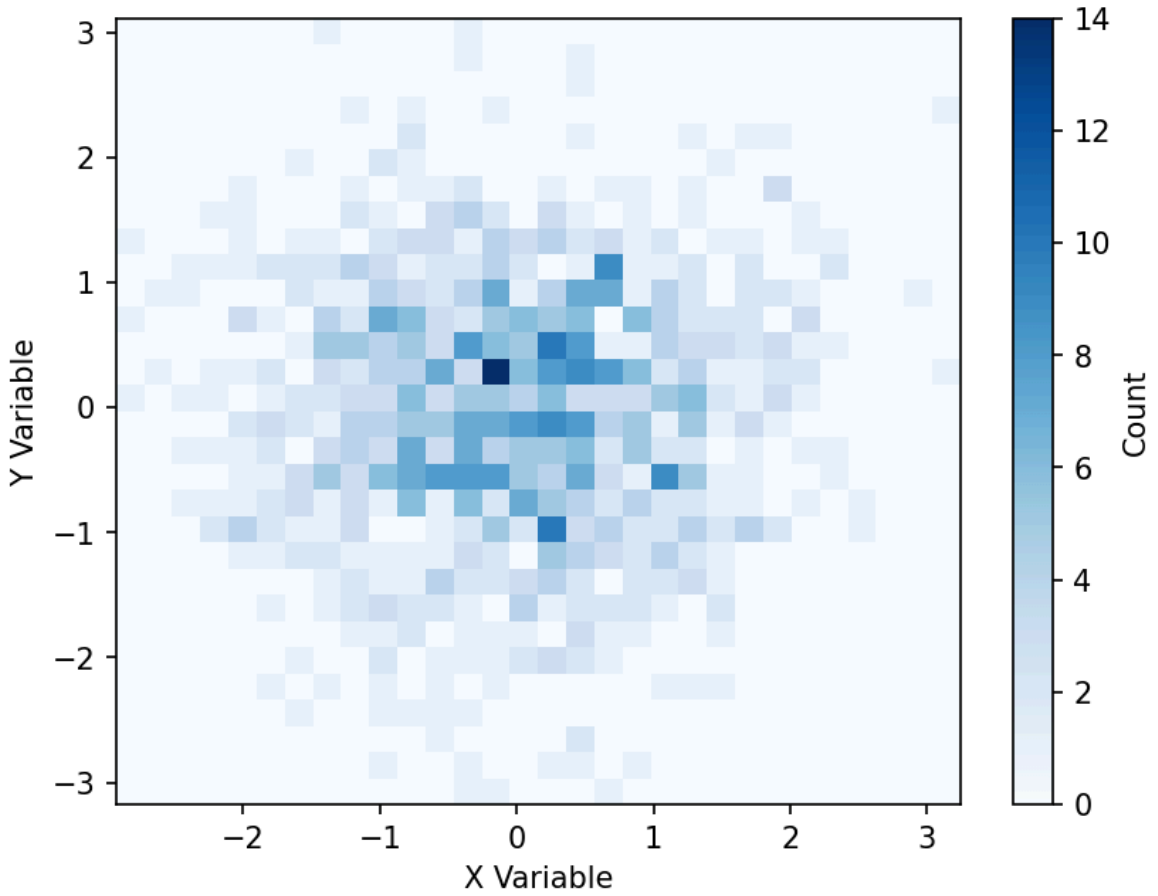
```python
# Multiple histograms (overlaid)
data1 = np.random.normal(15, 10, 1000)
data2 = np.random.normal(20, 8, 1000)

plt.hist(data1, bins=30, alpha=0.5, label='Group 1')
plt.hist(data2, bins=30, alpha=0.5, label='Group 2')
plt.legend()
plt.show()

# Cumulative histogram
plt.hist(data, bins=30, cumulative=True)
plt.ylabel('Cumulative Count')
plt.show()

# Horizontal histogram
plt.hist(data, bins=30, orientation='horizontal')
plt.show()

# 2D histogram (heatmap)
x = np.random.randn(1000)
y = np.random.randn(1000)
plt.hist2d(x, y, bins=30, cmap='Blues')
plt.colorbar(label='Count')
plt.xlabel('X Variable')
plt.ylabel('Y Variable')
plt.show()
```



Multiple Figures and Best Practices

Working efficiently with matplotlib

```python
# Multiple separate figures
plt.figure(1)
plt.plot([1, 2, 3], [1, 4, 9])
plt.title('Figure 1')

plt.figure(2)
plt.plot([1, 2, 3], [1, 2, 3])
plt.title('Figure 2')

plt.show()  # Shows all figures

# Subplots in one figure
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Plot in each subplot
axes[0, 0].plot(x, y)
axes[0, 0].set_title('Subplot 1')

axes[0, 1].scatter(x, y)
axes[0, 1].set_title('Subplot 2')

axes[1, 0].hist(data, bins=20)
axes[1, 0].set_title('Subplot 3')

axes[1, 1].bar(['A', 'B', 'C'], [1, 2, 3])
axes[1, 1].set_title('Subplot 4')

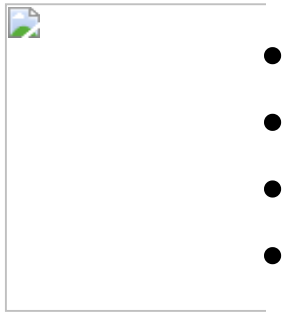
plt.tight_layout()
plt.show()
```

Best Practices:

- **Always label axes** with units
- **Use descriptive titles**
- **Add legends** for multiple datasets
- **Choose appropriate plot types** for your data
- **Use color-blind friendly palettes**
- **Save in vector formats** (PDF, SVG) for publications
- **Use tight_layout()** to prevent overlap
- **Close figures** when done: `plt.close()`

- Gallery: matplotlib.org/stable/gallery/
- Cheatsheet: matplotlib.org/cheatsheets/
- Documentation: matplotlib.org/stable/contents.html

Summary



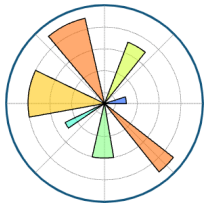
- Array creation and manipulation
- Indexing, slicing, and broadcasting
- Vectorized operations
- Linear algebra basics

Building your scientific computing toolkit

- **Practice** with real neuroscience data
- **Explore** the documentation and examples
- **Experiment** with different plot types
- **Combine** these tools in your research
- **Build** reproducible analysis pipelines



- Scientific computing ecosystem
- Specialized modules (stats, signal, optimize)
- When to use SciPy vs NumPy



- Basic plotting with pyplot
- Customization and styling
- Histograms and subplots

```
# Typical imports
import numpy as np
from scipy import signal, stats
```

Key Resources:

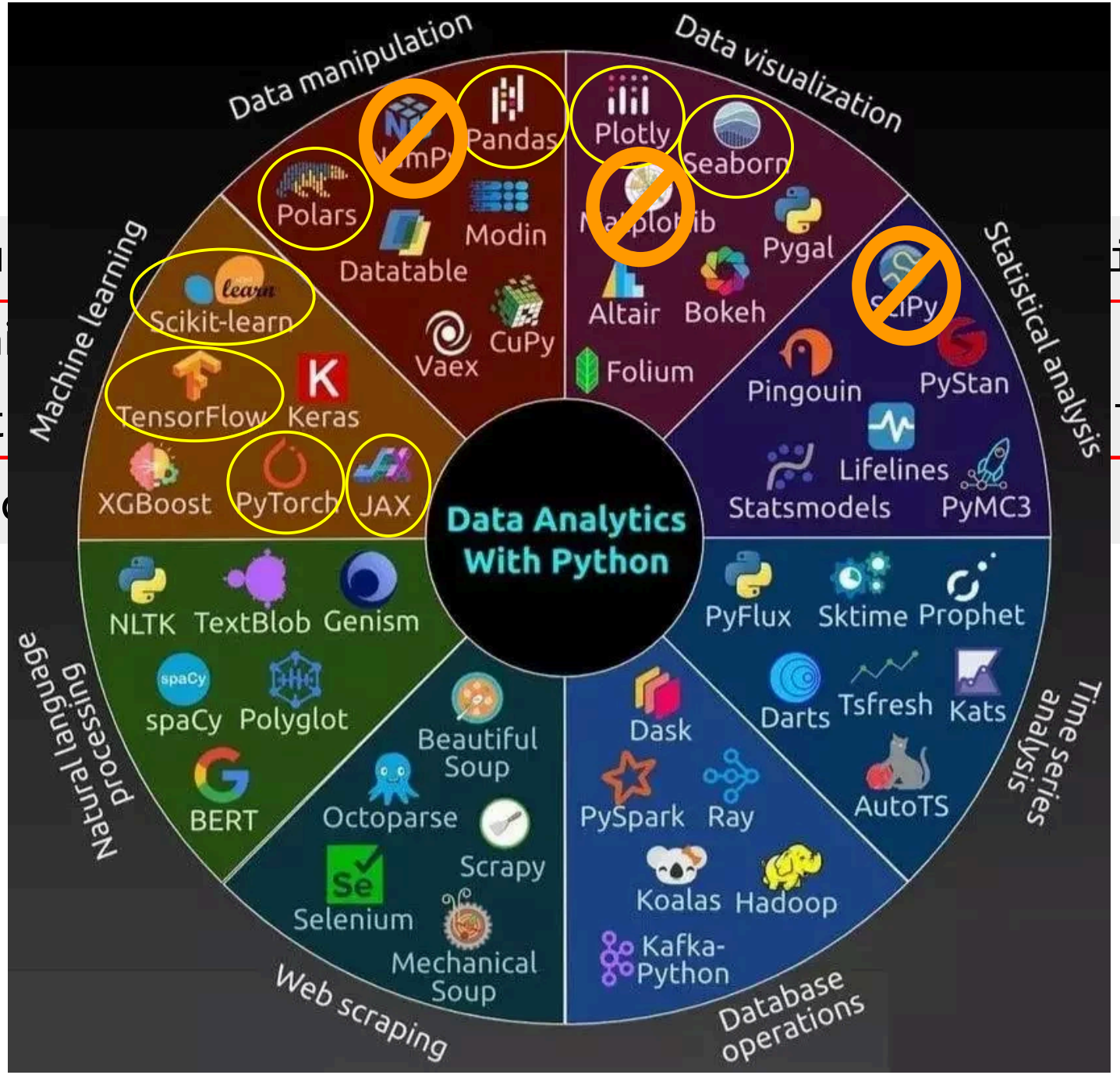
- NumPy: numpy.org
- SciPy: scipy.org
- matplotlib: matplotlib.org
- Gallery examples and tutorials

These three libraries form the foundation of scientific Python.

Master them, and you'll have the tools to tackle any data analysis challenge

Coming up

1. Numerica ib)
2. Data Mani
3. Visualizat ly)
4. Intermedi



Thank you!





Course materials (Part I) adapted from:

- Moritz Helias
- Abigail Morrison
- Yury Zaitsev
- Renato Duarte





With some help from Claude Sonnet 4.5

Claude